INTEGRAL DRAGON

MODULE 2 • SESSION 08

# Extending Capabilities with MCP

Connect the agent to tools without inventing a new integration every time

AI-Accelerated Software Development | Foundations Program

Confidential. Prepared exclusively for Ensemble Health Partners.

# Tool access changes what the agent can do — and changes what your team needs to govern

## WITHOUT TOOLS

- The model can reason about code it was shown
- It can draft, explain, and propose
- It cannot see live system state
- It cannot query databases, tickets, or repos in real time
- It cannot act on external systems

## WITH TOOLS VIA MCP

- Inspect live repository state
- Query databases directly by prompt
- Read tickets, docs, and telemetry
- Take actions through auditable integrations
- Work against the real environment, not a snapshot

**The implication:** More capability means a larger action surface. The same governance discipline that applies to human engineers — least privilege, audit trails, approved access — applies to the agent.

# MCP is infrastructure, not a chat feature — it belongs in enterprise architecture discussions

### The Model Context Protocol

An open standard for connecting LLM applications to external tools, resources, and prompts. Replaces brittle one-off custom wrappers with a shared protocol that any compliant client and server can speak.

### A Common Integration Contract

Teams no longer have to write custom glue code for every model-tool pairing. Build the MCP server once for your internal system, and any MCP-capable agent can connect to it through the same interface.

### Open Protocol Over Vendor Plugins

Vendor-specific plugin ecosystems create lock-in and fragmented governance. An open standard lets organizations reason clearly about capability boundaries, approve specific servers, and audit usage consistently.

### Why open protocols matter

- Less vendor lock-in — one standard, many providers
- Reusable integration patterns across teams
- Clearer boundary ownership and access control
- Easier enterprise governance than ad hoc plugin ecosystems

### MCP sits in architecture, not settings

It affects security boundaries, credential management, operational logging, and access control. Review it as part of your architecture process — not as a developer convenience decision.

# Servers, tools, resources, and prompts — the four things teams need to recognize

### MCP Server

The process that exposes capabilities — tools, resources, and prompts — to any connected MCP client. Your internal system (GitHub, database, ticketing) becomes an MCP server when you add the integration layer.

### Client

The application that connects to MCP servers and calls their capabilities on behalf of the model. Claude Code is the client in most enterprise contexts.

### Tools

Callable functions exposed by the server. The model invokes tools to take actions — querying a database, listing repo files, reading a ticket, updating a field. This is where capability and risk both live.

### Resources

Data that the server makes available for the model to read without direct function calls — documentation, schemas, configuration files, reference data.

### Prompts

Pre-structured prompt templates the server can provide. Allows teams to standardize how the model is instructed to use particular tools or interpret particular data.

### Transport

The message exchange mechanism between client and server. Determines how data moves and what operational choices exist around latency, reliability, and authentication.

# MCP should remove mystery, not add it — engineers must understand what a tool call is actually doing

### What the Payload Tells You

Every MCP tool call is structured data. It tells you which tool was called, which arguments were passed, what context the agent received back, and where authorization boundaries matter. This is legible by design.

### Inspectable Integrations

A good MCP integration is one engineers can inspect, explain, and audit. "Claude talked to the system somehow" is not an acceptable enterprise description of a production integration.

### Protocol Literacy Reduces Blind Trust

When teams understand the tool call structure, they can reason about risk. When they treat it as a black box, they extend trust without basis — which is how privileged access gets quietly over-exposed.

### Example tool call structure

```
// What a tool call looks like
{
  "tool": "list_issues",
  "arguments": {
    "repo": "org/payments-api",
    "label": "critical",
    "state": "open"
  }
}

// Questions to ask:
› What authority does list_issues carry?
› Can it see private repos?
› What credentials back this call?
› Is usage logged and reviewable?
```

# Tool access is a real capability boundary — treating every connector as harmless convenience is the enterprise mistake

## Security questions to ask immediately

- What authority does this server grant to the agent?
- Which credentials back the connection?
- Which tools should be exposed at all — and which should stay disabled?
- What is read-only vs. write-enabled?
- How is usage logged and reviewed?
- What is the rollout boundary for first exposure?

## The governance model

### Permissions

Specify exactly which tools a server may expose and what level of access each carries. Start with read-only. Expand only after governance review.

### Approved Servers + Scoped Credentials

Maintain a team or organization-level allowlist of approved MCP servers. Credentials should be scoped to the minimum access required — not a service account with elevated privileges.

### Auditable Usage

Every tool call should produce a log entry. Teams should be able to answer "what did the agent do through this integration and when?" without guessing.

LIVE DEMO — SESSION 08

# Connect one MCP server. Inspect one payload.

Read-only first. Understand what the connection actually created before expanding.

# A demo success is the beginning of the governance conversation — not the end of it

## Demo sequence

```
# Step 1: Connect a read-only server
$ claude mcp add github

# Step 2: List available tools
> "What tools does the GitHub server expose?"

# Step 3: Use one read-only action
> "Show open issues labeled 'critical'"
→ Live data returned via tool call

# Step 4: Inspect the payload structure
→ Walk through: tool name, arguments,
               returned fields, auth boundary
```

## The governance follow-up

```
# After the demo — the real questions:

› "Which tools from this server
    should remain disabled for now?"

› "What credentials back this connection
    and what scope do they carry?"

› "How would we audit usage of this
    tool set across the team?"

› "What is the safest first rollout
    boundary before broader access?"
```

**The principle:** Capability review is part of architecture review. Least privilege applies when the caller is an agent. Rollout should be staged, not assumed.

SESSION 08 — KEY TAKEAWAY

# MCP is the standard integration layer for agent tooling

That is exactly why teams must understand both the leverage and the guardrails.

**Next: Module 03** — Module 2 taught intent, plans, persistent rules, and connectivity. Module 3 builds on that foundation with hooks, skills, subagents, and orchestration.